Individual Technical Report

Helpdesk Ticketing System API

Prepared by:	Kawther Ali Almandhari
Role:	System Analysis, Team Discussion and Postman API Testing
Project Type:	Team Project - Day 1 / Individual Report - Day 2
Technology Stack:	Spring Boot, PostgreSQL, Hibernate/JPA, Maven, Postman, DbVisualizer
Date:	June 25, 2026

- This report focuses on my individual contribution in analysing the project, discussing issues with the team, and verifying the API behaviour through Postman testing evidence.

Table of Contents:

1. Executive Summary_ Introduction:

This individual report explains my contribution to the Helpdesk Ticketing System team project. The project involved reviewing an existing Spring Boot backend, identifying issues, fixing problems, improving the project where needed, and validating the required API operations. My main work focused on system analysis with the team and practical API testing using Postman. I tested the main endpoints, checked the request URLs and JSON bodies, compared the expected and actual outputs, and documented the results using screenshots.

2. Project Background_ Introduction:

The Helpdesk Ticketing System is a REST API used to manage support tickets. It includes users who raise tickets, agents who handle tickets, SLA rules that define resolution targets, ticket status changes, comments, and reporting endpoints. The backend uses Spring Boot with PostgreSQL and JPA/Hibernate. The project had to be functional, documented, and tested clearly as required by the team project instructions.

3. My Individual Contribution _Body:

My contribution was mainly in three areas: analysing the project behaviour, participating in team discussions, and carrying out Postman-based testing. During the analysis stage, I helped check how the endpoints were expected to work and what data was required before testing. During team discussions, I raised problems found during testing, especially errors related to missing database records and invalid IDs. During the testing stage, I executed requests in Postman and verified whether the HTTP status codes and JSON responses were correct.

4. Challenges Faced

One challenge was understanding the correct testing sequence. For example, a ticket could not be created before a user and an SLA rule existed. Another challenge was identifying whether an API response was an actual bug or an expected business-rule response. For example, HTTP 409 Conflict looked like an error at first, but it was correct when the system rejected an invalid ticket status transition. A third challenge was making sure the database was properly prepared before continuing with Postman tests.

5. Issues and Bugs Identified

Several issues appeared during testing. First, the application needed a correct database configuration in application.properties before the API could connect to PostgreSQL. Second, ticket creation failed when the required SLA rule was missing from the database. Third, some requests failed because an incorrect ticket ID or user ID was used. Fourth, invalid ticket status transitions were rejected with HTTP 409 Conflict. This was not a bug; it confirmed that workflow validation was working. These findings helped the team understand what needed to be fixed or prepared before final verification.

6. How Issues Were Resolved

The database connection was verified through the Spring Boot configuration. The missing SLA data was resolved by inserting default SLA rules using DbVisualizer. After that, ticket creation succeeded because the required slaRuleId existed. Invalid ID problems were handled by checking existing records first, such as using GET /api/users and GET /api/tickets before assigning or updating a ticket. Business-rule conflicts were accepted as correct results when they matched the intended workflow rules.

7. Technical Decisions Made by the Team

The team continued using Spring Boot because it provides a clean structure for REST API development. PostgreSQL was used as the database because the project required relational data such as users, agents, tickets, comments,

and SLA rules. Postman was selected for API testing because it clearly shows request methods, URLs, JSON bodies, status codes, and response payloads. DbVisualizer was used to inspect and update database records when required.

8. API Testing Evidence Using Postman

The following section presents the practical testing evidence. Each screenshot shows the request and output from either the development environment, DbVisualizer, or Postman. The purpose of including screenshots is to show that the API was tested directly and that the outputs were reviewed against the expected behaviour.

API Testing Summary

Test	Endpoint	Expected Output	Validation
Create User	POST /api/users	201 Created	Correct - user saved successfully
Create Agent	POST /api/agents	201 Created	Correct - agent saved successfully
Create Ticket	POST /api/tickets?userId=5&slaRuleId=2	201 Created	Correct after SLA data was inserted
Assign Ticket	POST /api/tickets/{id}/assign?agentId=1	200 OK	Correct when valid ticket and agent IDs were used
Update Status	PUT /api/tickets/{id}/status?status=...	200 OK / 409 Conflict	Correct - valid transitions succeed and invalid ones are rejected
Filter Tickets	GET /api/tickets?...	200 OK	Correct - returns matching tickets
Average Resolution Time	GET /api/tickets/metrics/avg-resolution-time	200 OK	Correct - returns metric value
Overdue Tickets	GET /api/tickets/overdue	200 OK	Correct - empty list is valid when no ticket is overdue

9. Suggestions for Improvement

- Add Swagger documentation so the API can be tested from the browser.
- Add authentication and role-based access for users, agents, and administrators.
- Add pagination and sorting for ticket lists.
- Improve error handling so duplicate emails and missing resources always return clear 400/404/409 responses.
- Add email notifications when a ticket is assigned, resolved, or closed.
- Add a simple SLA management API so SLA rules can be created without manual SQL insertion.

10. Lessons Learned

- A backend API should be tested in the correct order because some endpoints depend on existing records.
- HTTP status codes are important evidence; not every non-200 response is a bug.
- Database preparation is part of API testing, especially when business rules depend on existing data.
- Postman is useful for verifying request methods, request bodies, response codes, and JSON outputs.
- Team discussion helps separate real bugs from expected business-rule behaviour.

11. Conclusion

Overall, the Helpdesk Ticketing System API was analysed, tested, and verified successfully. My work focused on understanding the project flow, discussing technical issues with the group, and validating the API through Postman. The screenshots included in this report show the actual testing evidence and explain why each output was correct or, in some cases, why it appeared as an error during testing. The project now has clear evidence that the main API operations work as required.

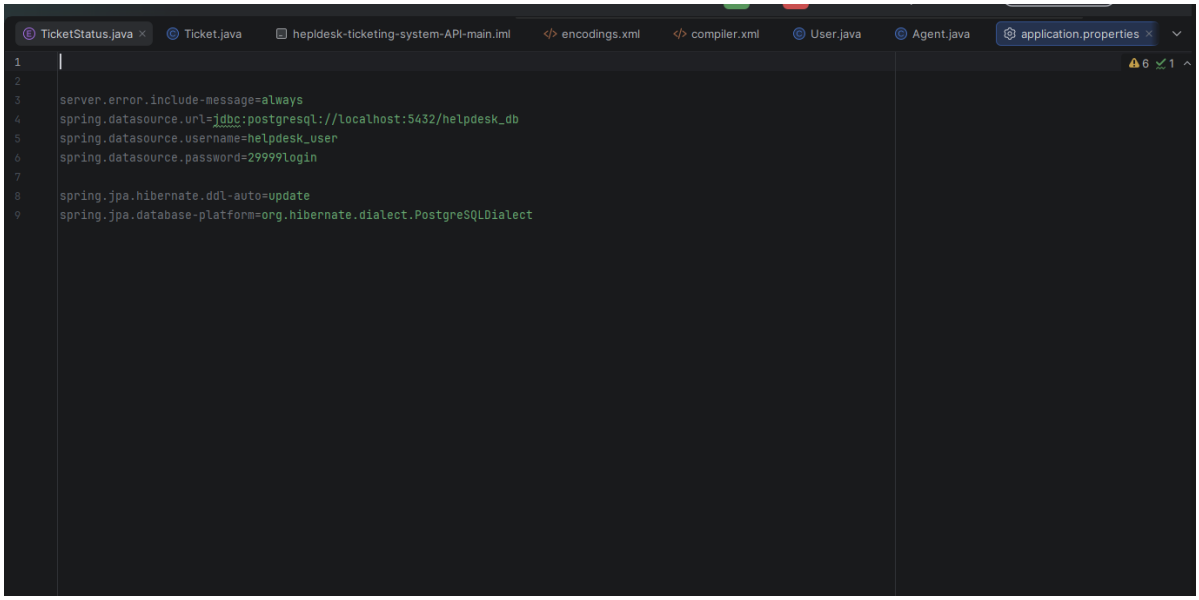
References

Project source code and local testing environment: Spring Boot Helpdesk Ticketing System API.

Tools used: Postman, DbVisualizer, Docker, IntelliJ IDEA, PostgreSQL.

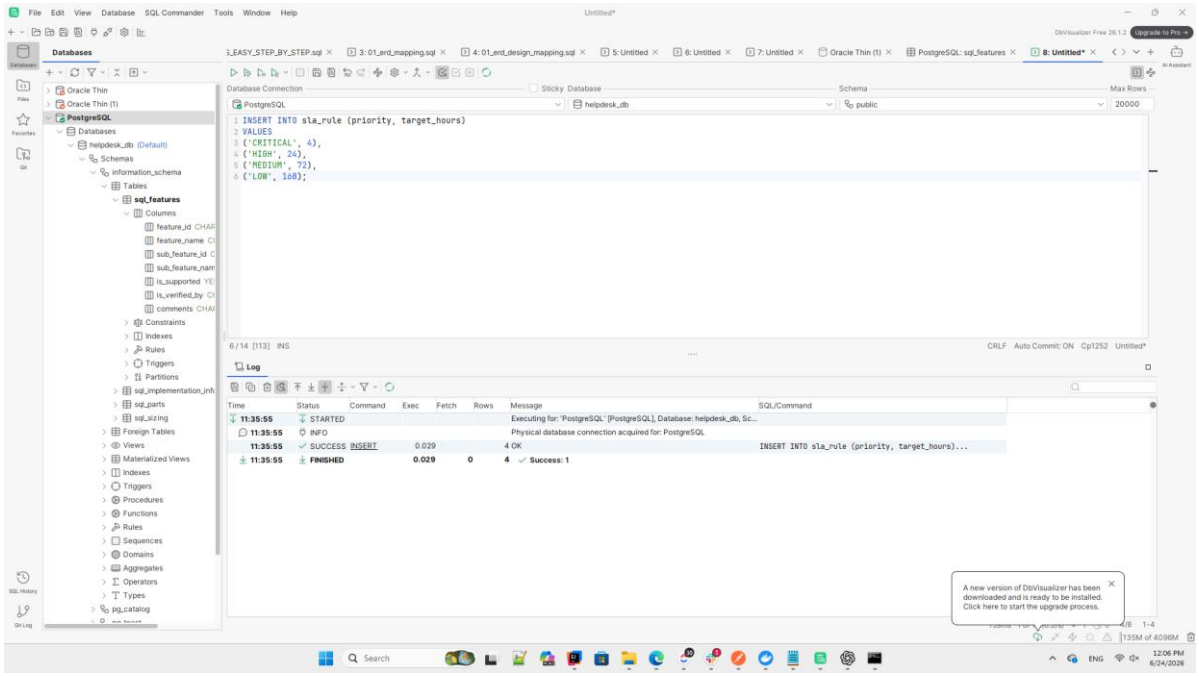
Appendix A. Testing Screenshots

Figure 1. Spring Boot database configuration



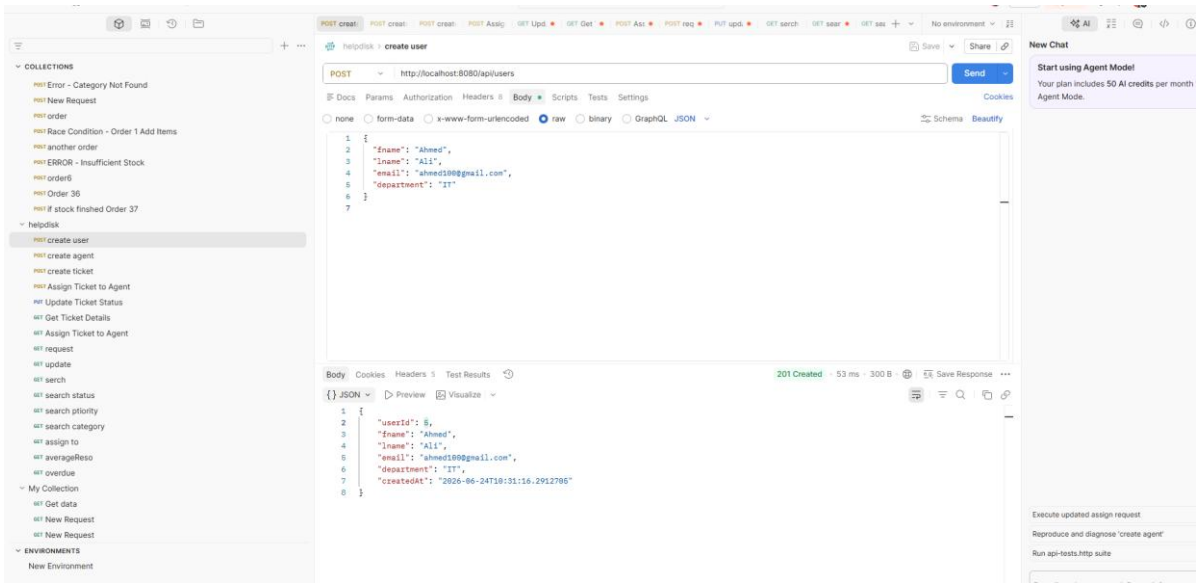
Explanation: This screenshot shows the application.properties configuration used to connect the Spring Boot API to PostgreSQL. The database URL points to helpdesk_db, Hibernate is set to update the schema during development, and detailed error messages are enabled to support API testing.

Figure 2. SLA rule insertion in DbVisualizer



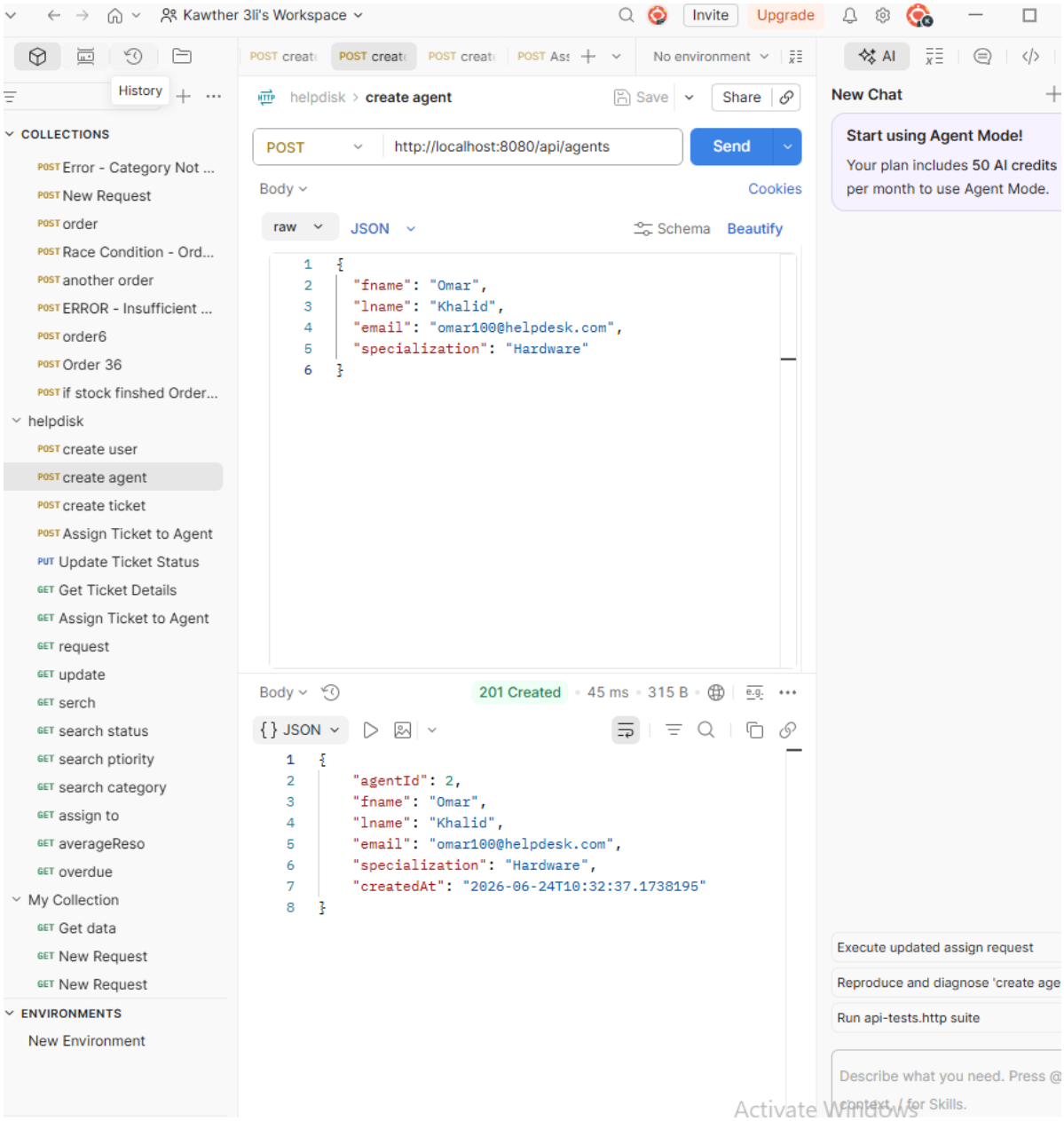
Explanation: This screenshot shows the SQL insert used to add the missing SLA rules. This was required because ticket creation depends on an existing SLA rule. The successful execution confirmed that the database had the records needed before Postman testing continued.

Figure 3. Create User request



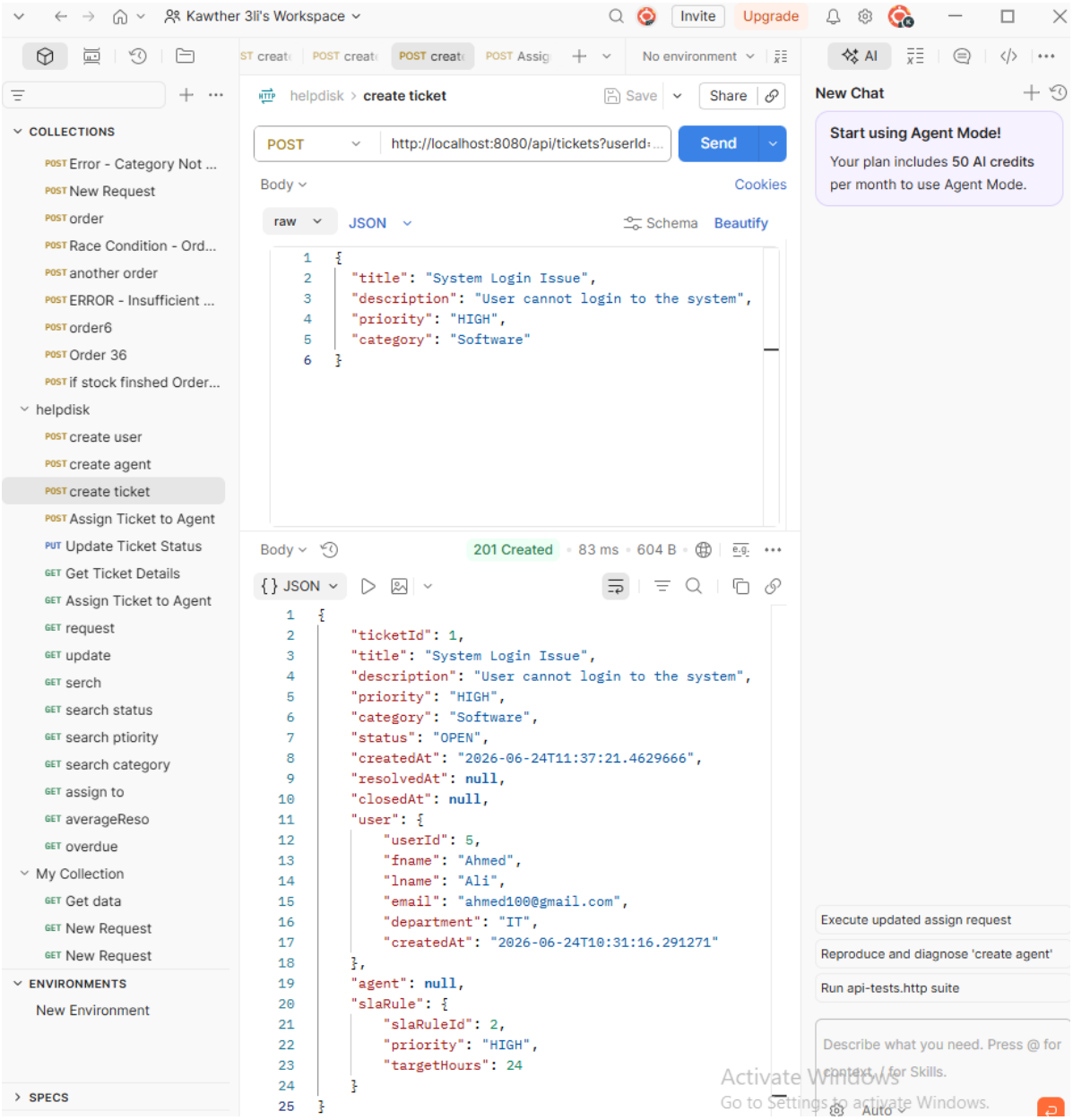
Explanation: The Create User endpoint was tested using POST /api/users. The response confirmed that the user was created successfully and stored in the database. This was required before creating a ticket because every ticket must be linked to a user.

Figure 4. Create Agent request



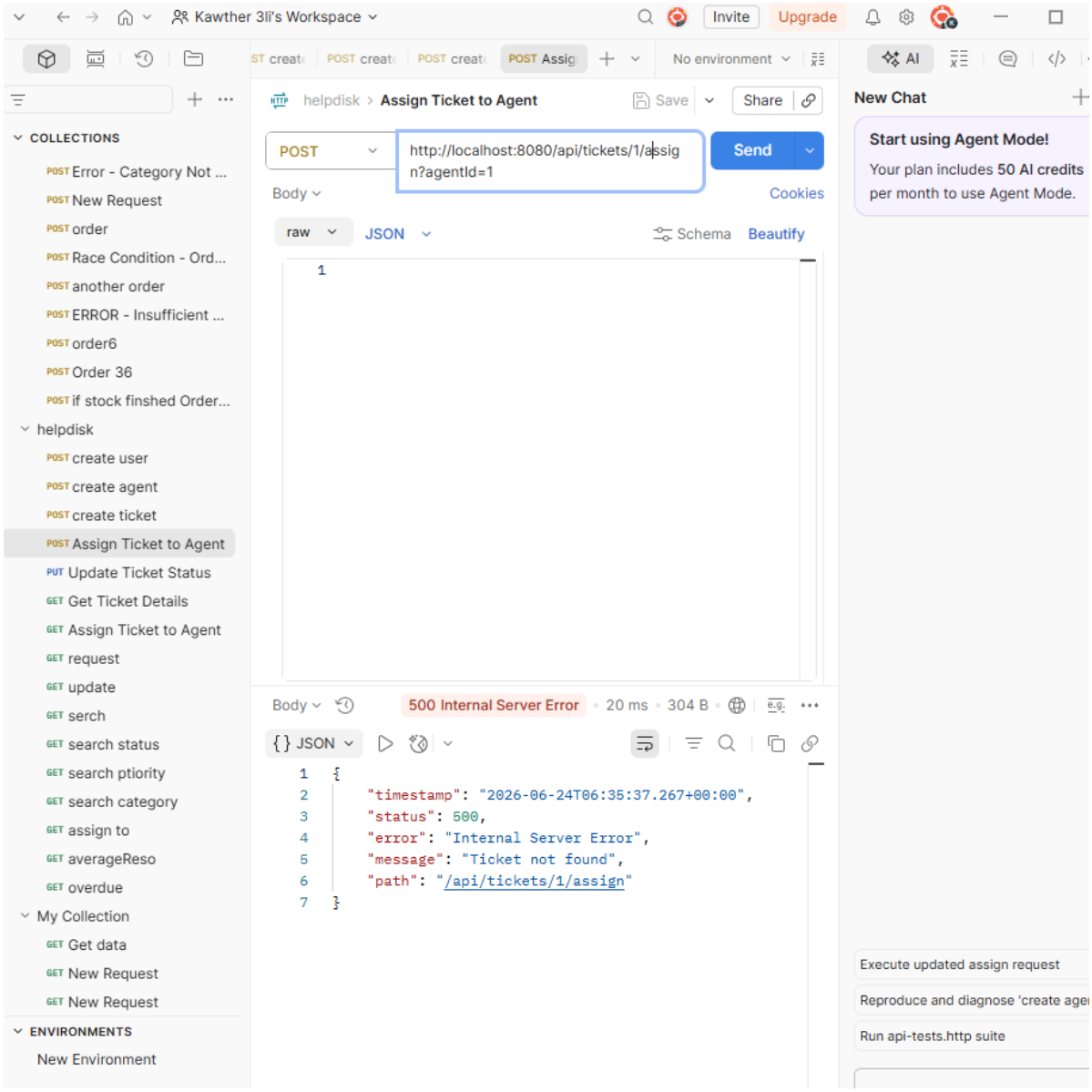
Explanation: The Create Agent endpoint was tested using POST /api/agents. The successful response confirmed that the support agent was saved and could later be assigned to a ticket.

Figure 5. Create Ticket request



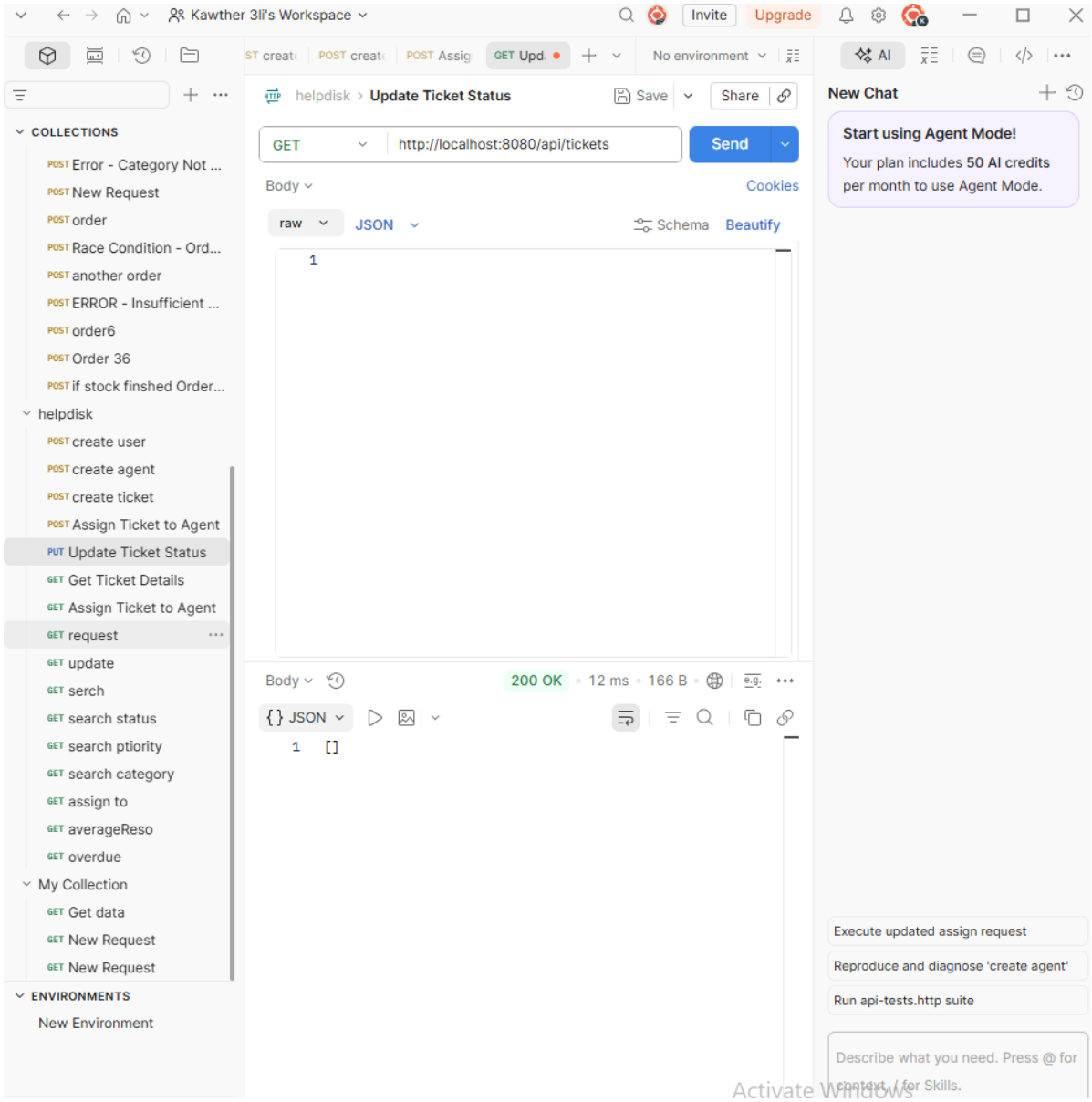
Explanation: The Create Ticket endpoint was tested after preparing the user and SLA rule records. The response showed a new ticket with OPEN status, which is the correct initial state for the workflow.

Figure 6. Ticket assignment issue



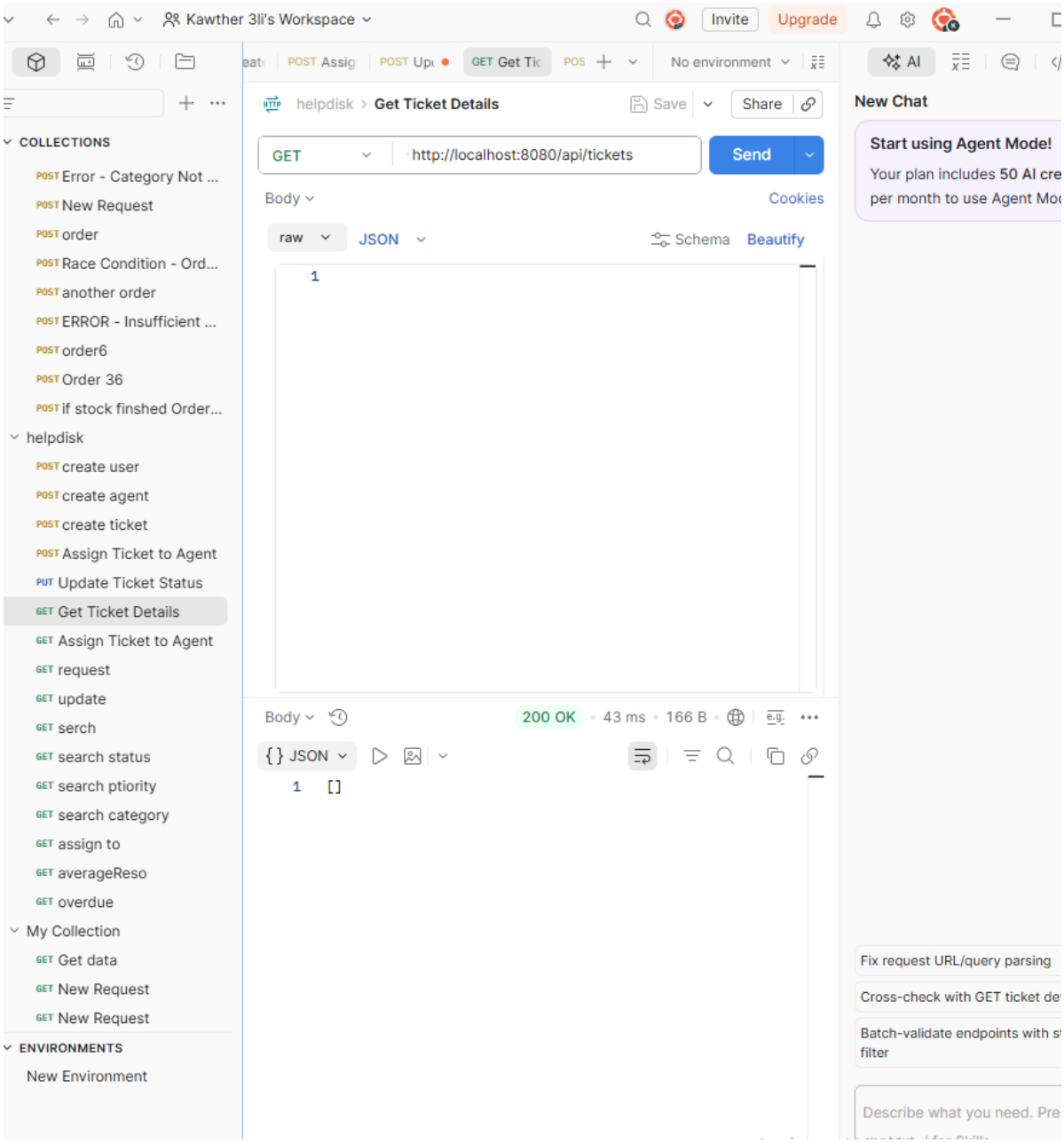
Explanation: This screenshot shows an error response during ticket assignment. The output was useful because it helped identify that the request was using an invalid ticket reference. This was treated as a testing issue rather than a successful system behaviour.

Figure 7. Empty ticket list response



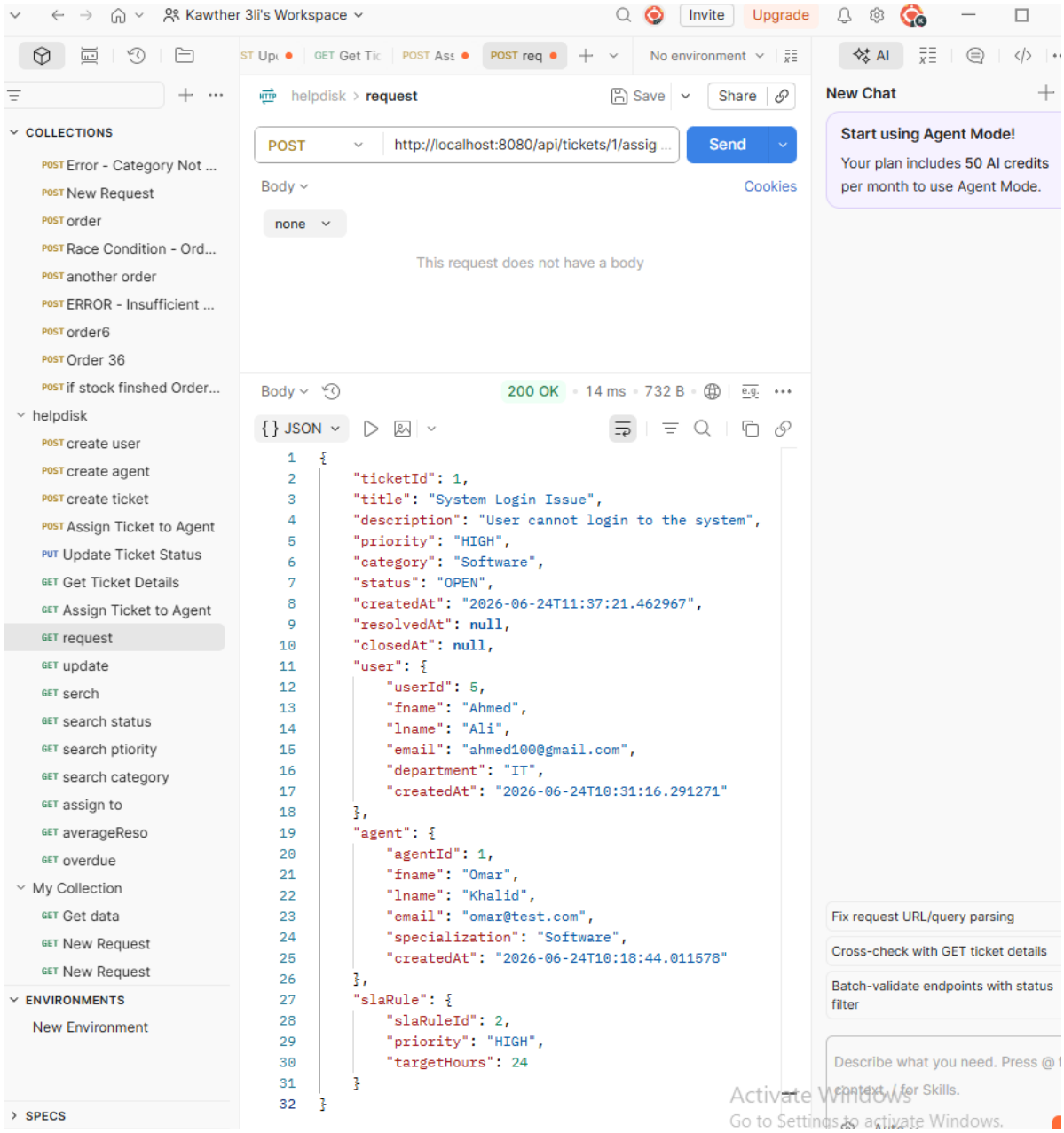
Explanation: This GET request returned an empty list. The response is acceptable when no matching ticket records exist yet or when the filter does not match the stored data.

Figure 8. Get ticket detail / empty result check



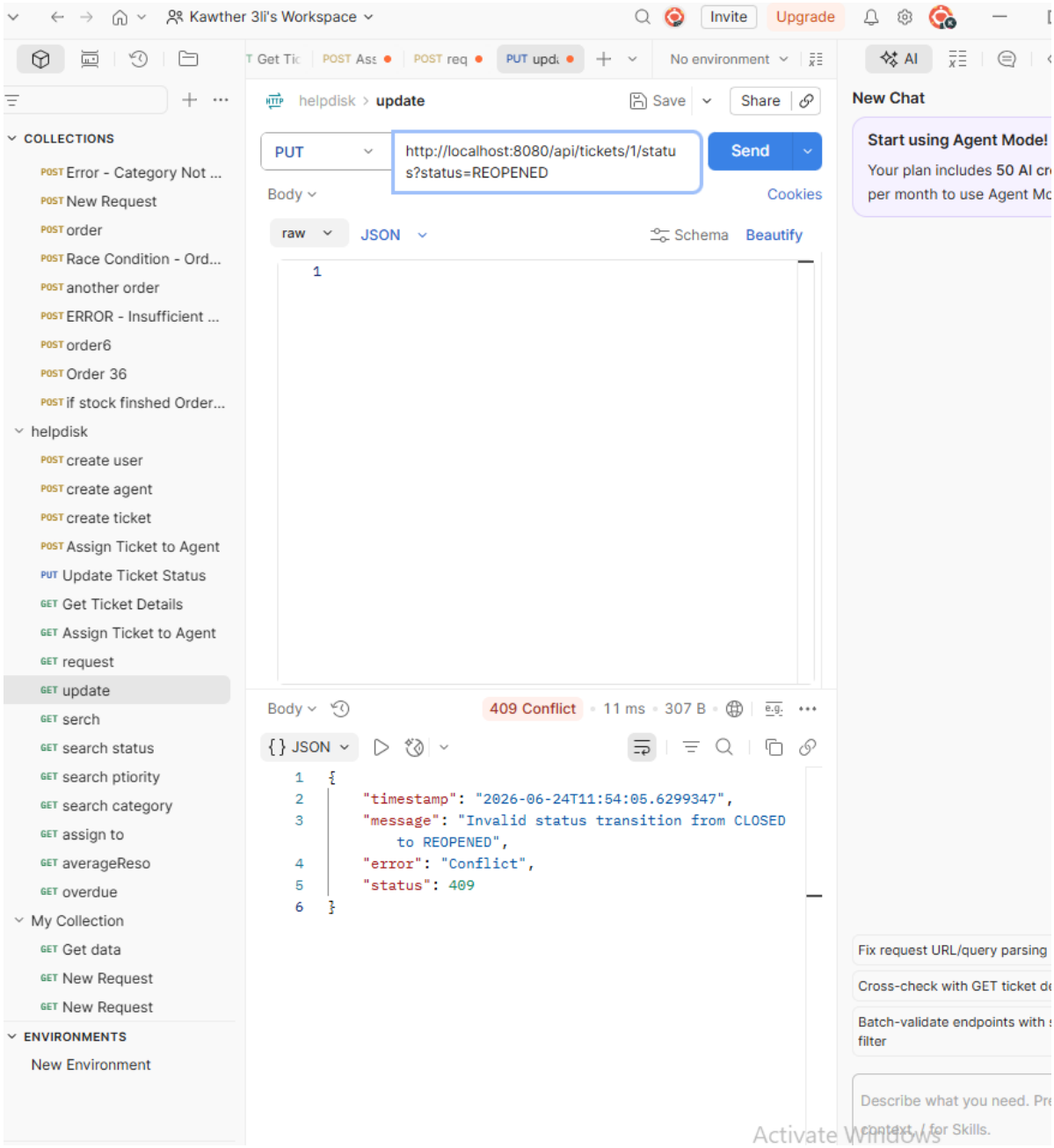
Explanation: This test was used to check ticket retrieval behaviour. An empty response indicates that the requested data was not available at that stage, which helped confirm the importance of creating and using the correct ticket ID.

Figure 9. Assign Ticket success



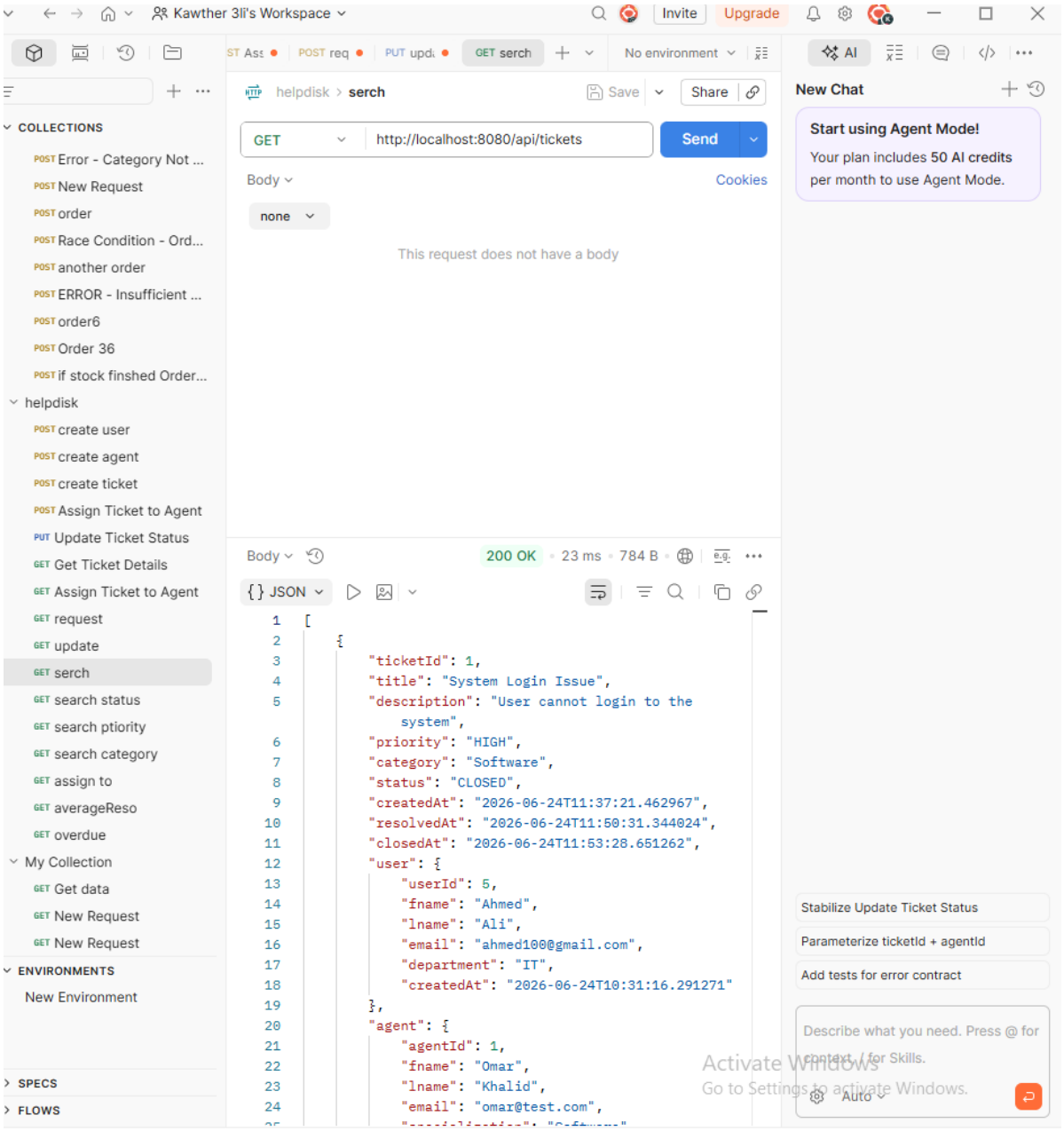
Explanation: The ticket assignment request succeeded after using the correct ticket and agent references. The response includes agent information, which confirms that the ticket was assigned successfully.

Figure 10. Invalid status transition response



Explanation: This test returned HTTP 409 Conflict because the requested status transition was not allowed by the ticket workflow rules. This is a correct response because the system should reject invalid business operations.

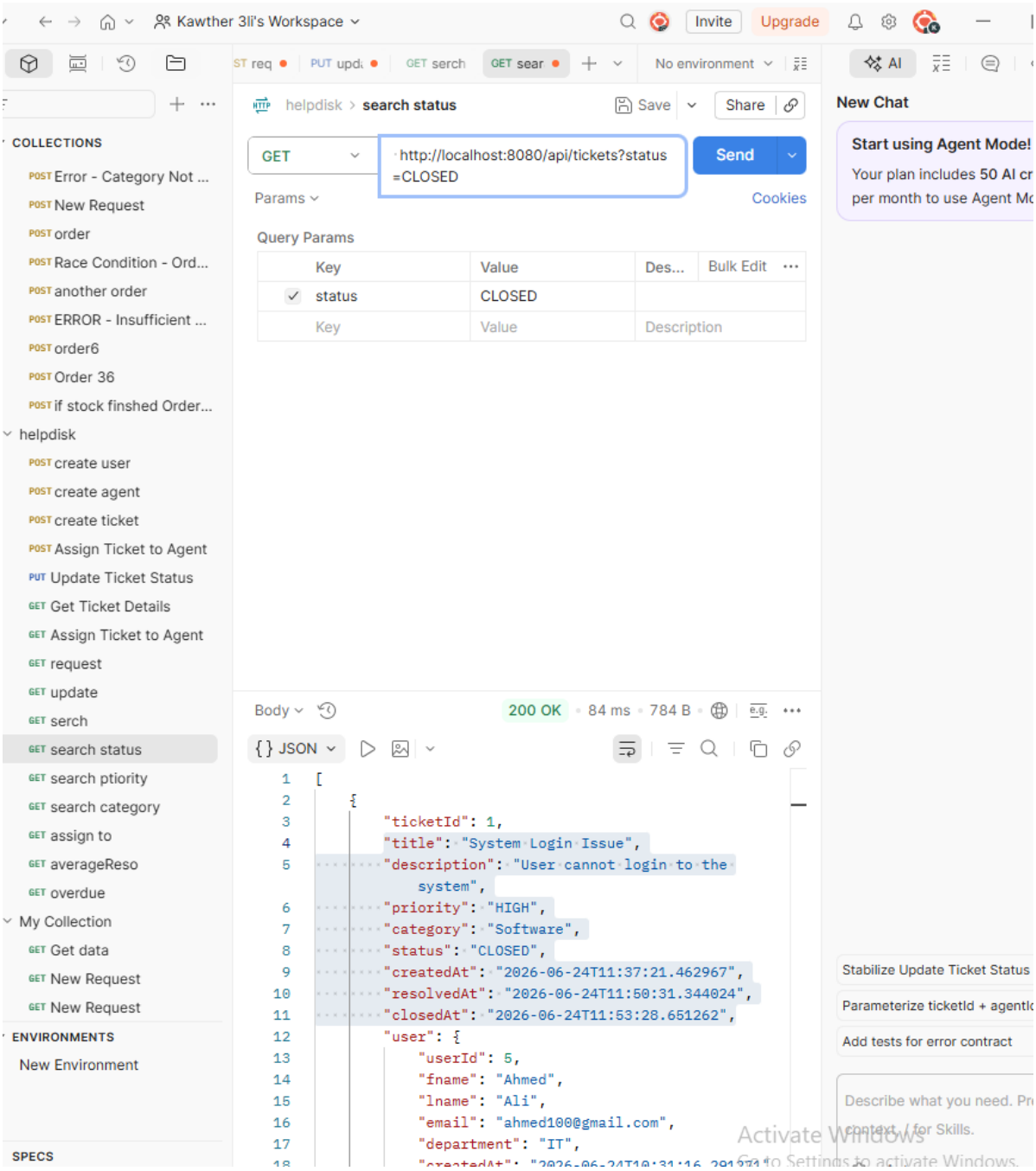
Figure 11. Get all tickets




```
[
  {
    "ticketId": 1,
    "title": "System Login Issue",
    "description": "User cannot login to the system",
    "priority": "HIGH",
    "category": "Software",
    "status": "CLOSED",
    "createdAt": "2026-06-24T11:37:21.462967",
    "resolvedAt": "2026-06-24T11:50:31.344024",
    "closedAt": "2026-06-24T11:53:28.651262",
    "user": {
      "userId": 5,
      "fname": "Ahmed",
      "lname": "Ali",
      "email": "ahmed100@gmail.com",
      "department": "IT",
      "createdAt": "2026-06-24T10:31:16.291271"
    },
    "agent": {
      "agentId": 1,
      "fname": "Omar",
      "lname": "Khalid",
      "email": "omar@test.com",
```

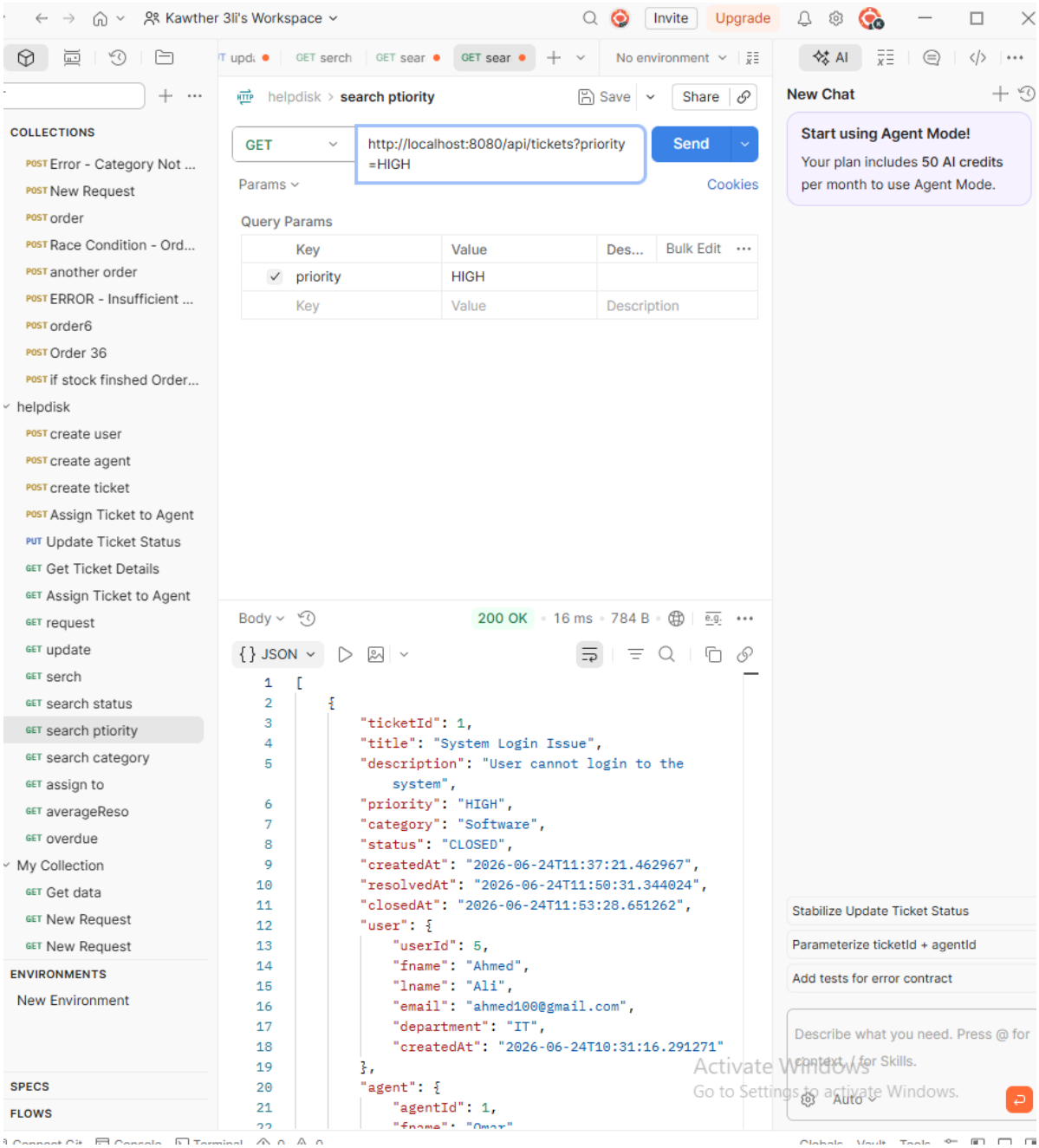

Explanation: The GET /api/tickets request returned the available ticket records. This confirmed that the ticket data could be retrieved after creation and assignment.

Figure 12. Filter tickets by status



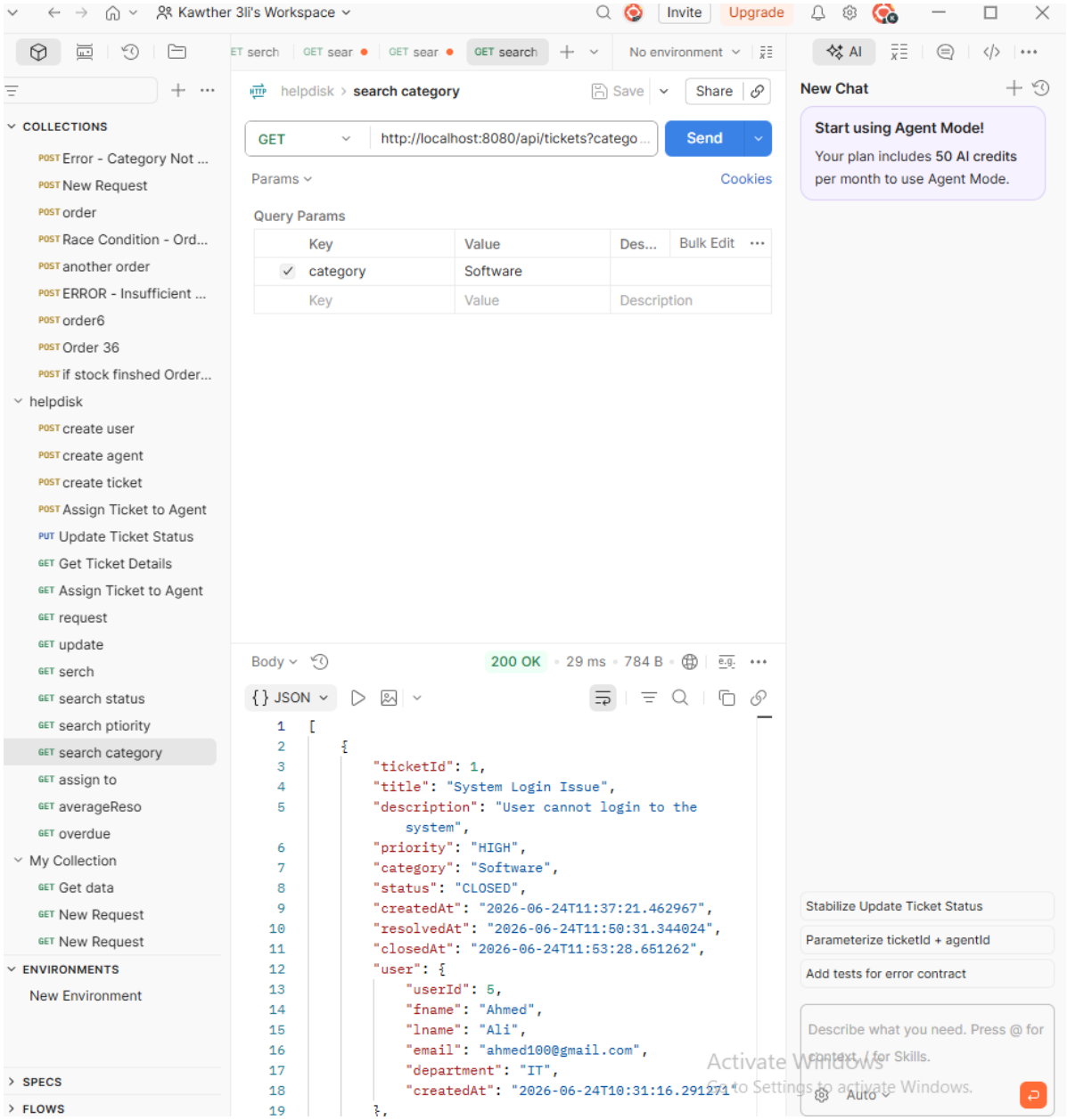
Explanation: The status filter was tested to confirm that tickets could be retrieved based on their current workflow state. This supports monitoring and operational reporting.

Figure 13. Filter tickets by priority



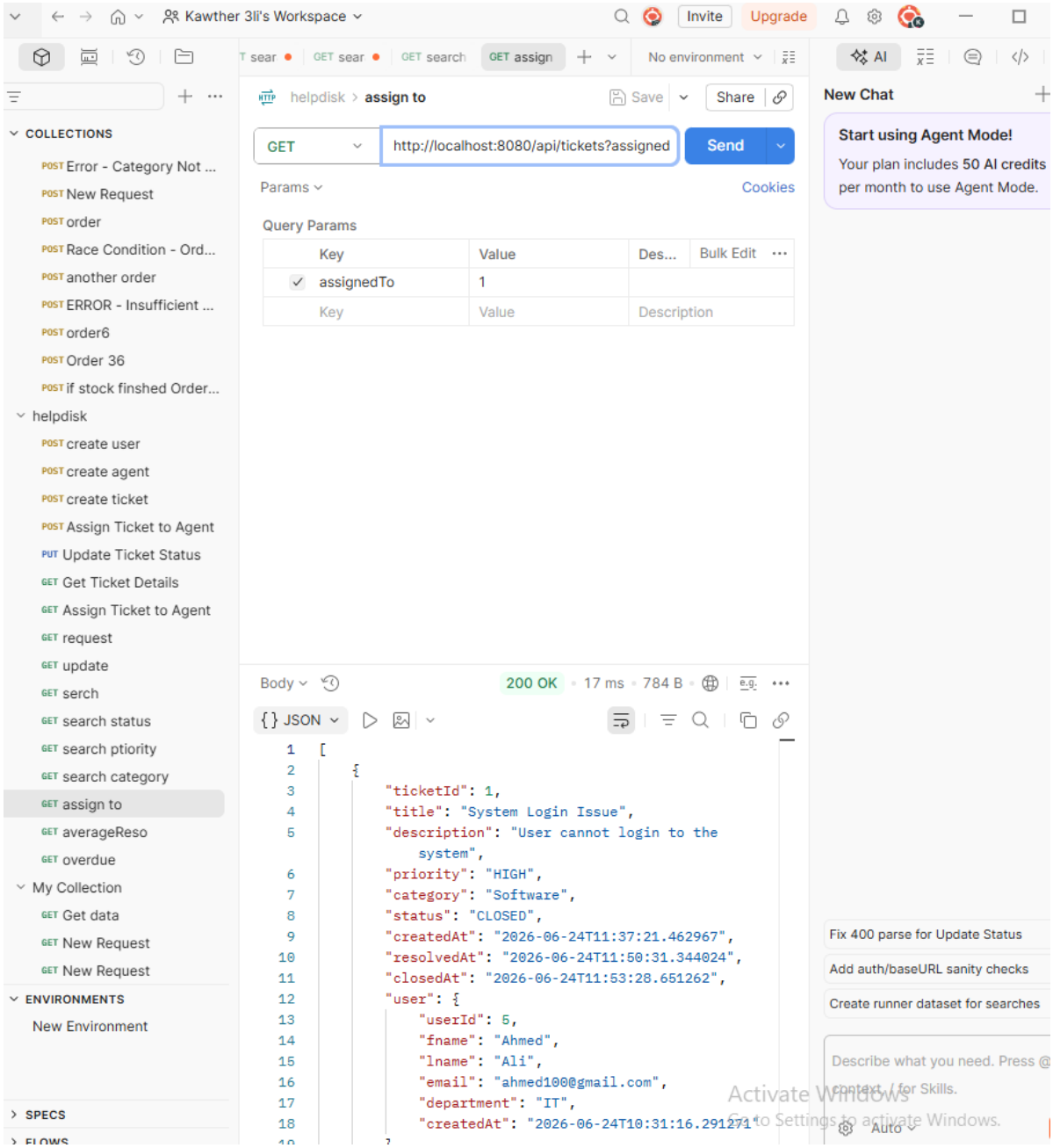
Explanation: The priority filter was tested to confirm that HIGH priority tickets could be retrieved correctly. This is important because priority is linked to SLA handling.

Figure 14. Filter tickets by category



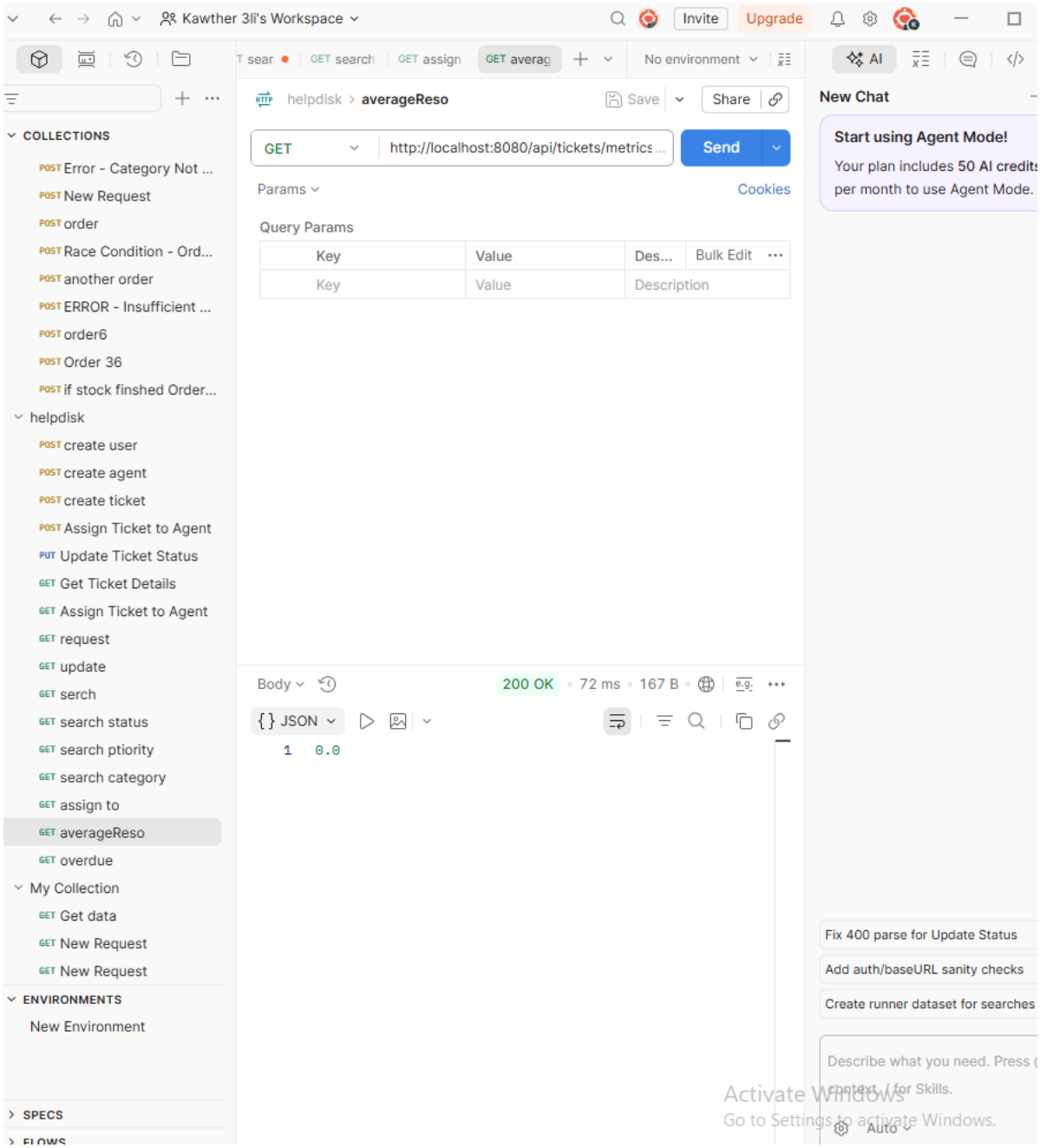
Explanation: The category filter was tested using the Software category. The result confirmed that the API supports category-based ticket retrieval.

Figure 15. Filter tickets by assigned agent



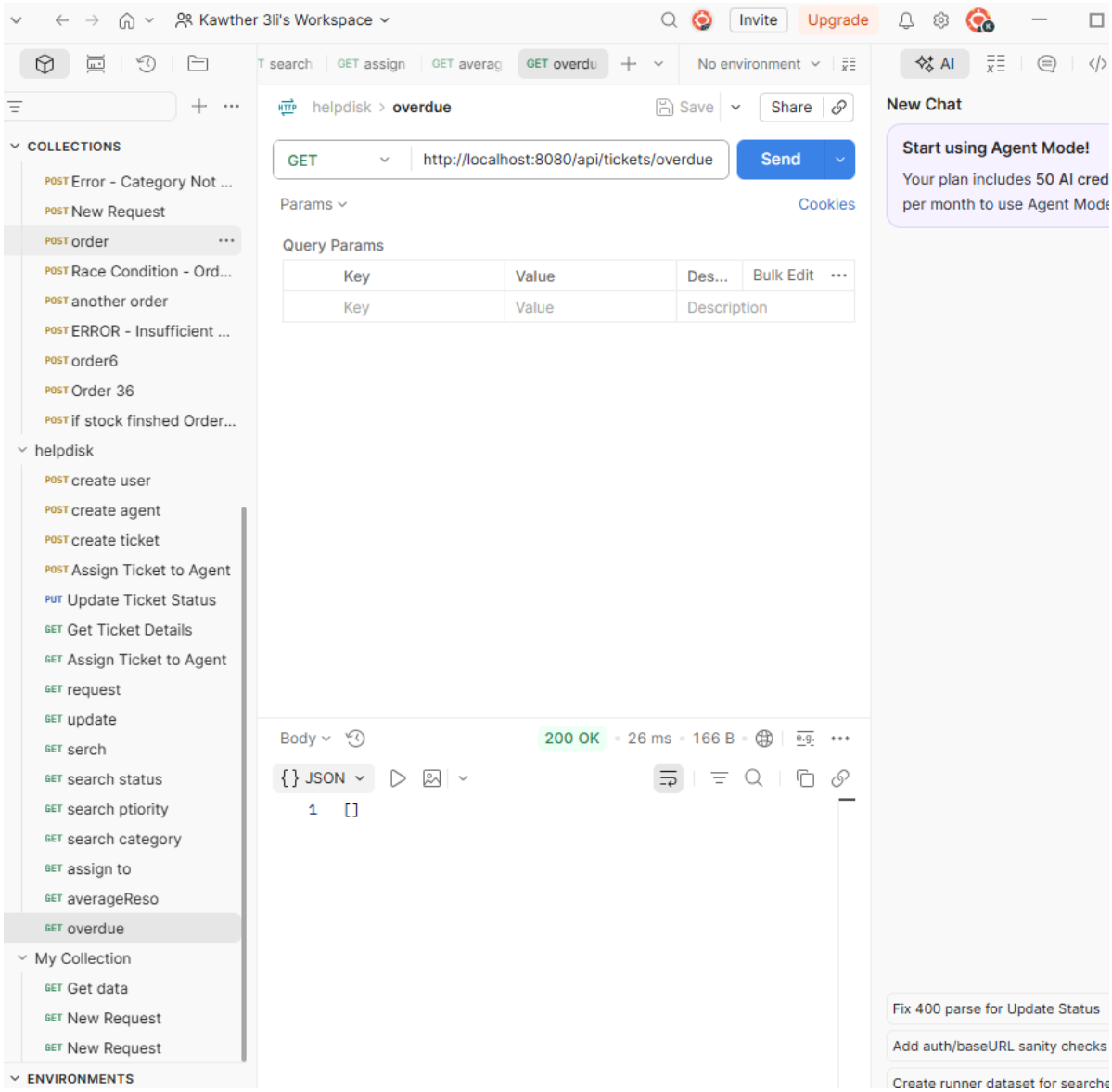
Explanation: This request tested filtering tickets by the assigned agent. The response confirms that tickets can be tracked by support ownership.

Figure 16. Average resolution time endpoint



Explanation: The average resolution time endpoint returned 0.0. This is acceptable when there is not enough resolved ticket duration data available, or when the tested data does not produce a measurable average.

Figure 17. Overdue tickets endpoint



Explanation: The overdue tickets endpoint returned an empty list. This is correct when no ticket has exceeded its SLA deadline.